IMEN574 / IDSC574 · PROGRAMMING FOR AX

6

Arrays and DataFrames

Sangwoo Mo · POSTECH
Spring 2027

Where We Are

01 Programming with AI

Lectures 2–4

02 **Python and Classical Machine Learning**

Lectures 5–13

03 Deep Learning and Foundation Models

Lectures 14–22

Outline: Arrays and DataFrames

01 **Compute With Axes**

Name the numerical dimensions

02 **Work With Labels**

Preserve the observation identity

03 **Combine With Checks**

Verify row units and keys

01 · Compute With Axes

01 **Compute With Axes**

Name the numerical dimensions

02 **Work With Labels**

Preserve the observation identity

03 **Combine With Checks**

Verify row units and keys

Why an Array Shape

Correct arithmetic can still use the wrong axis

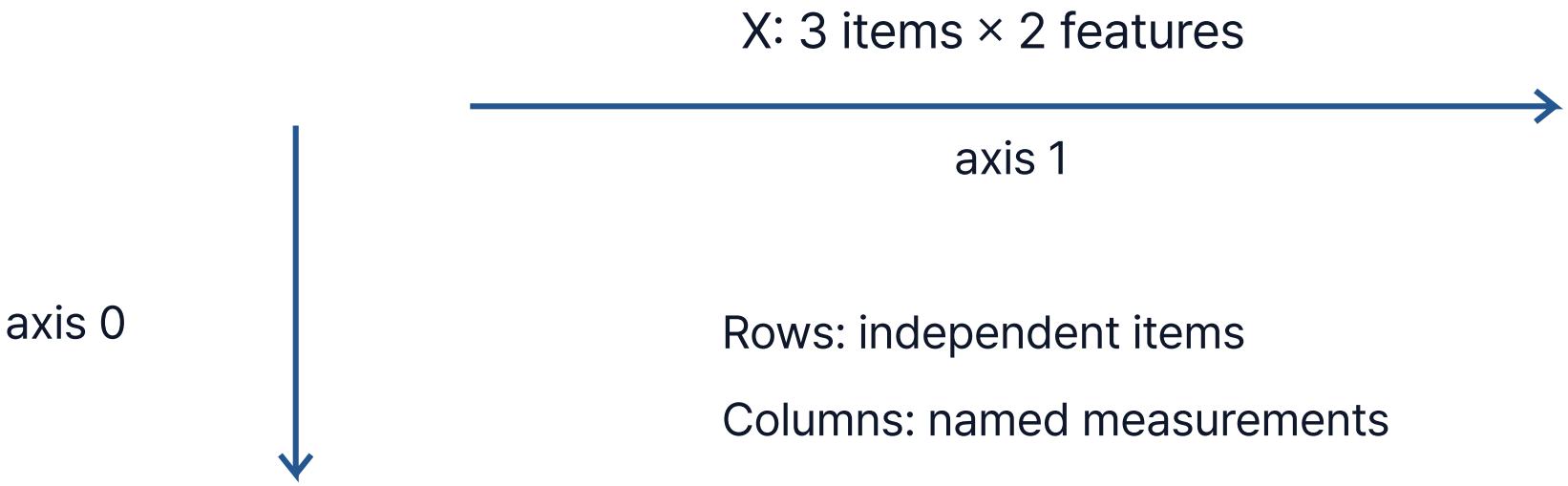
	Length	Mass
Item A	10	100
Item B	20	200
Item C	30	300

```
X.shape = (3, 2)
```

Three rows and two features define different aggregation axes

Dimensions and Axes

Write down what each axis means



Rows: independent items

Columns: named measurements

Axis labels make the intended reduction explicit

Array Construction

Choose a numeric representation deliberately

```
import numpy as np
X = np.array([[10, 100], [20, 200], [30,
300]], dtype=float)
print(X.shape)
print(X.dtype)
```

```
(3, 2)
float64
```

Shape and dtype are part of the numerical contract

Indexing and Slicing

Select by position before computing

Selection	Result
X[0]	[10,100]
X[:,0]	[10,20,30]
X[:2,1]	[100,200]

Each index addresses one axis

Reduction Over Rows

Axis zero combines the items

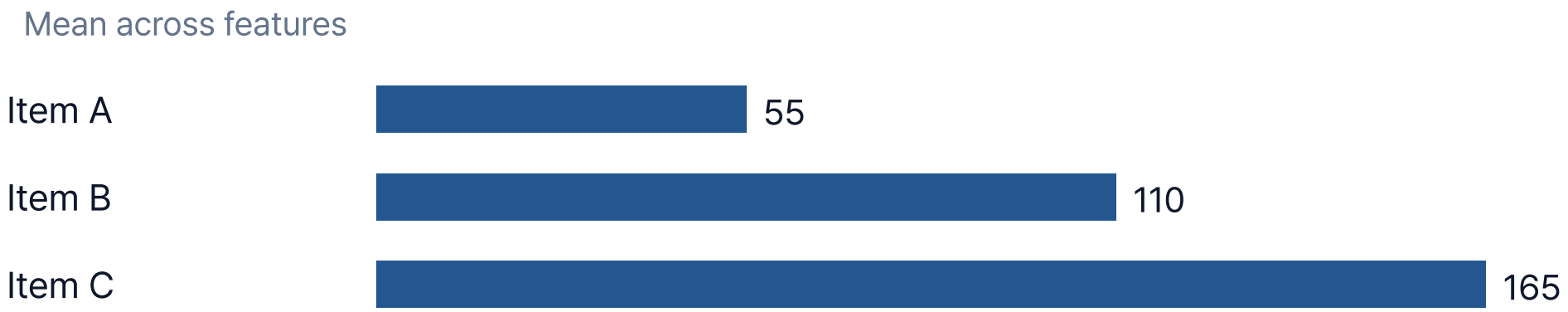
	Length	Mass
Item A	10	100
Item B	20	200
Item C	30	300
Mean	20	200

```
X.mean(axis=0) = [20, 200]
```

Combining three items leaves one value per feature

Reduction Over Features

Axis one combines unlike measurements



A valid reduction can answer a meaningless question

Reshape and Meaning

The same values can be arranged into different axes

ORIGINAL	RESHAPED
[1 2 3]	[1 2]
[4 5 6]	[3 4]
shape (2,3)	[5 6]
	shape (3,2)

Reshaping preserves values, not the interpretation of axes

Broadcasting Rules

Compare dimensions from the trailing axis

Array	Axis 0	Axis 1
X	3	2
offset	1	2
result	3	2

```
(3, 2) + (2,) → (3, 2)
```

The feature offset is shared across all rows

https://numpy.org/doc/stable/user/basics.broadcasting.html

Rowwise and Columnwise Offsets

Add the intended vector along the intended axis

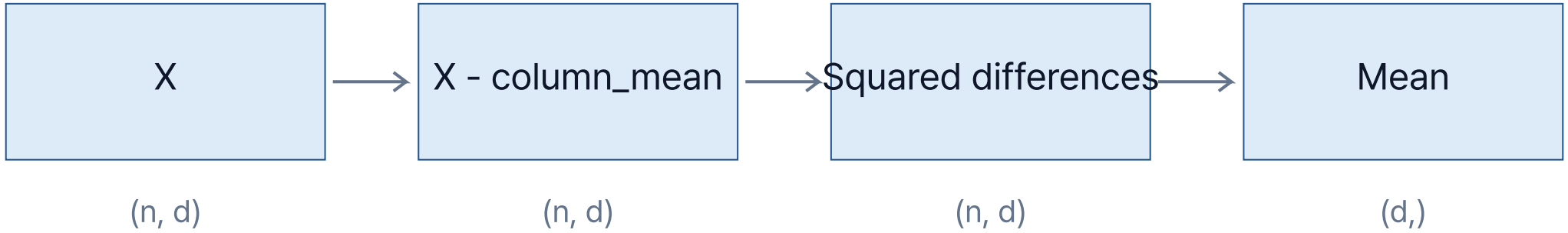
```
import numpy as np
X = np.array([[10,100],[20,200],[30,300]])
print(X + np.array([1,2]))
print(X + np.array([1,2,3])[:,None])
```

```
[[ 11 102]
 [ 21 202]
 [ 31 302]]
[[ 11 101]
 [ 22 202]
 [ 33 303]]
```

An added singleton axis changes the broadcast direction

Vectorized Computation

Express a whole numerical operation at once



The intermediate shapes explain the numerical operation

Caveat: Views and Copies

A slice can share the original storage

```
import numpy as np
X = np.array([1,2,3,4])
part = X[:2]
part[0] = 99
print(X)
print(X[:2].copy())
```

```
[99  2  3  4]
[99  2]
```

Changing a view changes its shared array

02 · Work With Labels

01 **Compute With Axes**

Name the numerical dimensions

02 **Work With Labels**

Preserve the observation identity

03 **Combine With Checks**

Verify row units and keys

Why a Labelled Table

A positional result needs stable row identities

Item ID	Length	Mass
p03	10	100
p08	20	200
p11	30	300

The item key survives changes in row order

Series and DataFrames

A column is a labelled one-dimensional object

DATAFRAME	SERIES
Rows × columns	One labelled column
df[["length"]]	df["length"]
Shape (3,1)	Shape (3,)

Selecting one column can preserve or remove an axis

Label and Position

loc and iloc answer different indexing questions

Address	First	Second	Third
Index	p03	p08	p11
Position	0	1	2

```
df.loc["p08"] = df.iloc[1]
```

A label and a position are different addresses

Boolean Filtering

A predicate selects rows that meet a condition

Item	Length	Length >= 20
p03	10	False
p08	20	True
p11	30	True

The predicate keeps two identified rows

Column Transformations

Keep the unit in the feature name

```
import pandas as pd
df = pd.DataFrame({"length_mm":[10,20,30]})
df["length_cm"] = df["length_mm"] / 10
print(df.to_string(index=False))
```

```
 length_mm  length_cm
         10        1.0
         20        2.0
         30        3.0
```

A named output column documents the unit change

Missing Values

A missing measurement is not a measured zero

Item	Length
p03	10
p08	missing
p11	30

```
mean(observed) = 20 ≠ mean([10,0,30])
```

Zero imputation changes the meaning of the measurement

Alignment by Label

Pandas aligns labels before adding values

Label	Left	Right	Sum
a	10	1	11
b	20	missing	missing
c	missing	2	missing

Only the shared label has both operands

GroupBy as a Unit Change

Aggregation turns rows into groups

	Line	Length
Item 1	A	10
Item 2	A	20
Item 3	B	30

```
group mean: A → 15, B → 30
```

Three item rows become two line summaries

Aggregation and Transform

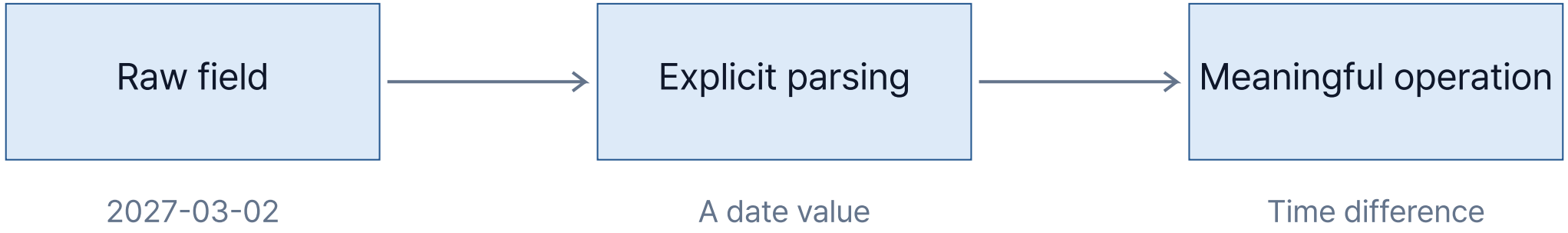
Choose the output row unit first

AGGREGATE	TRANSFORM
A → 15	Item 1 → 15
B → 30	Item 2 → 15
One row per group	Item 3 → 30
	One row per original item

Transform preserves the original item count

Dates and Categories

Parse semantics before comparing text



A representation choice constrains valid operations

Caveat: Duplicate Keys

A repeated identity may be an extra observation or an error

Item ID	Time	Length
p03	08:00	10
p03	09:00	11
p08	08:00	20

An item key alone does not identify a measurement

03 · Combine With Checks

01 **Compute With Axes**

Name the numerical dimensions

02 **Work With Labels**

Preserve the observation identity

03 **Combine With Checks**

Verify row units and keys

Why Join Validation

A merge can silently multiply the rows

Item	Line
p03	A
p08	B

```
2 item rows × duplicate lookup keys → 3 result rows
```

Key multiplicity determines the result size

One-to-One and Many-to-One

A lookup key should have the declared multiplicity

ITEMS	LOOKUP
p03 → A	A → North
p08 → A	B → South
p11 → B	Unique line key

A unique lookup key preserves the item row count

A Cardinality Check

Ask merge to validate the assumed key relation

```
result = items.merge(
    lookup, on="line", how="left",
    validate="many_to_one", indicator=True
)
assert len(result) == len(items)
print(result["_merge"].value_counts())
```

The merge checks multiplicity and unmatched rows

Unmatched Rows

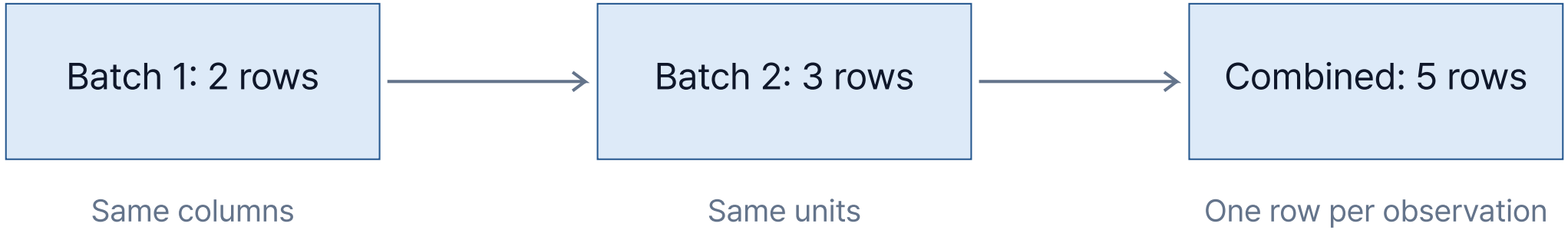
A left join retains an item with no lookup value

Item	Line	Region
p03	A	North
p08	C	missing

Preserving rows does not guarantee matched attributes

Concatenation

Stack compatible observations along a declared axis



Stacking needs compatible columns and units

Pivoting

Change the layout while keeping the data unit explicit

LONG FORM	WIDE FORM
A, day1, 10	Item day1 day2
A, day2, 12	A 10 12
B, day1, 20	B 20 missing

A cell is identified by an item-day pair

Rolling Windows

A window specifies which past rows contribute

Time	Value	Two-Row Mean
t1	10	-
t2	20	15
t3	30	25
t4	40	35

A trailing window uses a defined set of rows

Demo: Feature Scaling

Change one axis and inspect the nearest item

Scale one feature; identify which item becomes nearest

Y scale 1



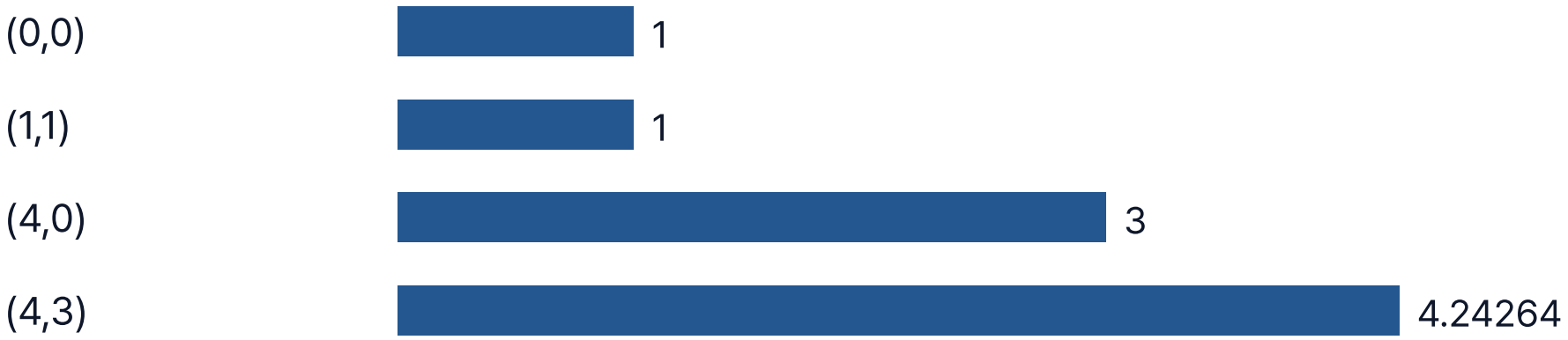
Scaled distance from (1,0): P0=1.00, P1=1.00, P2=3.00, P3=4.24

Nearest point P0; y is multiplied by 1

Answer: Unit Scaling

The query is one unit from two points

```
q = (1, 0), d(q, (0,0)) = 1, d(q, (1,1)) = 1
```



The default input contains a genuine nearest-neighbor tie

Caveat: Elementwise or Matrix

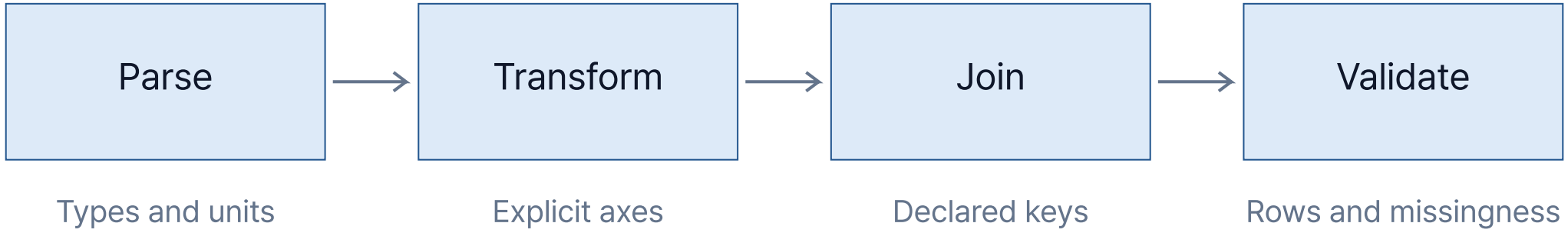
Two similar operators express different computations

ELEMENTWISE	MATRIX PRODUCT
X * X	X.T @ X
Square each cell	Combine rows by multiplication
Shape stays (n,d)	Shape becomes (d,d)

The operator determines the meaning of multiplication

A Checked Table Pipeline

Keep assertions near the operation they protect



Each step preserves a stated data assumption

Takeaway: Arrays and DataFrames

01 Name axes before reducing arrays

02 Track labels and observation units

03 Validate key multiplicity after a join